
NemHem Unified AI Interface – Detailed Documentation

Project Overview

The **NemHem Unified AI Interface** is a Streamlit-based web application designed to allow seamless interaction with **multiple LLM providers** like:

-  Google Gemini
-  Together.ai
-  Groq
-  Mistral AI
-  OpenRouter

It supports **chatting with individual models** or **chaining multiple models**, where the output of one is passed to the next — enabling complex reasoning or multiple perspective responses.

Key Features

Feature	Description
 Multi-LLM Support	Interact with Gemini, Together, Groq, MistralAI, and OpenRouter in one UI
 Chain Mode	Automatically passes output from one model to the next
 Model Switcher	Choose your preferred model from a dropdown
 Chat History	View all previous messages in the current session
 Secure API Key Storage	Uses a <code>.env</code> file to store credentials safely

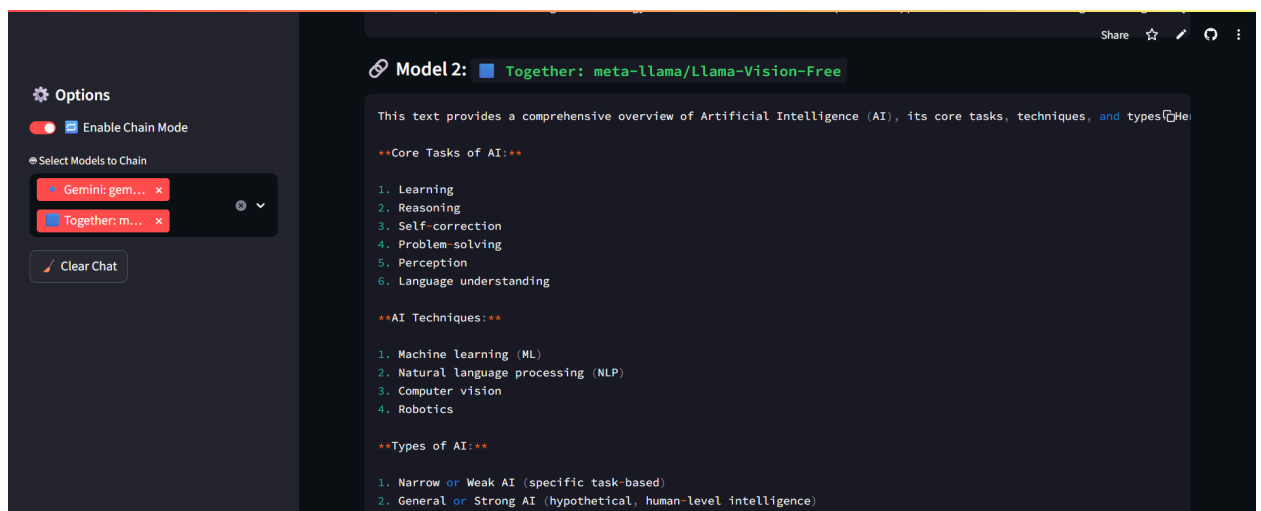
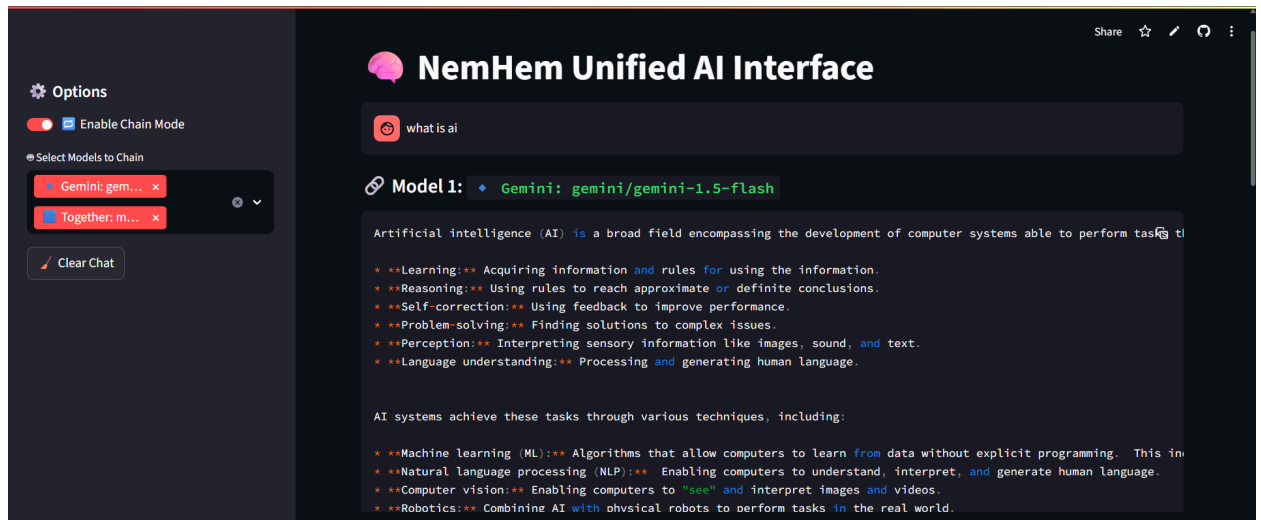


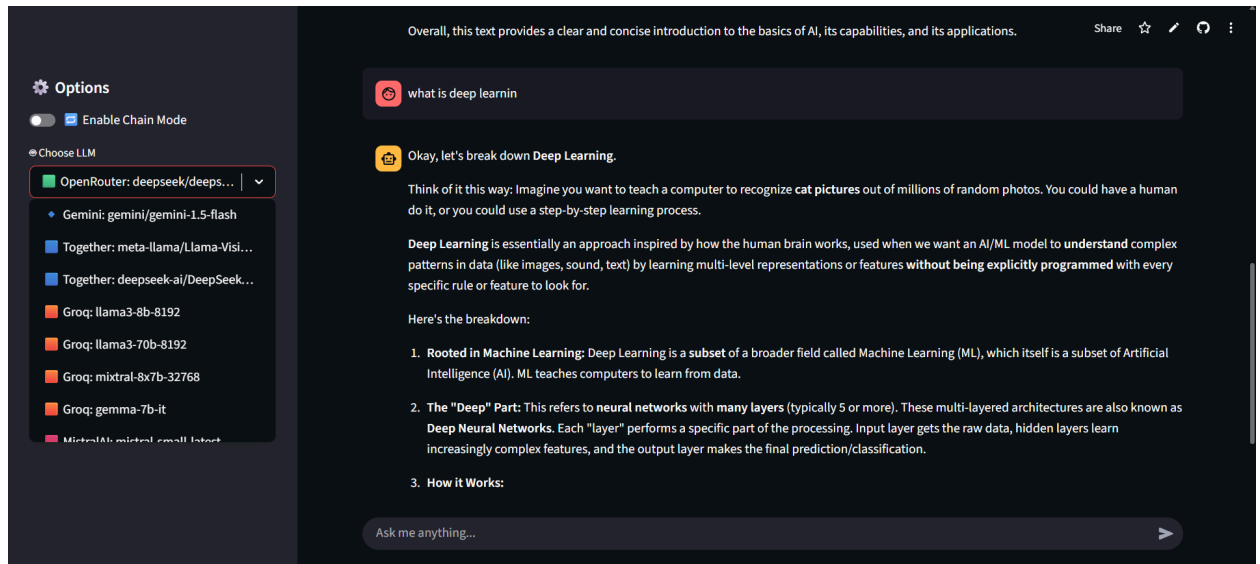
Easy Deployment

Can run locally or deploy to cloud via Streamlit Cloud, Render, etc.



User Interface Preview





Prerequisites

Make sure you have:

- Python 3.8+
- Pip
- API keys for the models you want to use (Gemini, Together, Groq, etc.)

Setup Instructions

Step 1: Clone the Repository

Shell

```
git clone <your-repo-url>
cd <repo-folder>
```

Step 2: Install Python Dependencies

Shell

```
pip install -r requirements.txt
```

Example `requirements.txt`:

None

```
streamlit
python-dotenv
requests
google-generativeai
together
```

Step 3: Configure `.env` File

Create a `.env` file in your root directory and add the API keys:

None

```
# Gemini
GEMINI_API_KEY=your_gemini_api_key

# Together
TOGETHER_API_KEY=your_together_api_key

# Groq
GROQ_API_KEY=your_groq_api_key

# OpenRouter (comma-separated if multiple)
OPENROUTER_API_KEYS=your_openrouter_api_key

# Mistral
MISTRAL_API_KEY=your_mistral_api_key
```

 *Do not commit this file to GitHub.*

Step 4: Run the Application

Shell

```
streamlit run app.py
```

 [Insert Screenshot: Running App Example]

File Structure

None

```
project-root/
|
├─ app.py           # Main Streamlit app
├─ .env             # API keys (excluded from Git)
├─ requirements.txt # Python libraries
└─ uploads/         # Optional file upload support
```

Chaining Logic Explanation

In **Chain Mode**, users can select multiple models. The app:

1. Sends the user's prompt to the first model.
2. Takes the **output of the first** model as **input to the second**.
3. Repeats this process until the final model produces a response.

 [Insert Diagram: Prompt → Model 1 → Output → Model 2 → Output → ...]

Example Use Case

- **Use Case:** You ask: *"Explain Quantum Computing simply."*
 - **Model 1:** Gemini generates a general overview.

- **Model 2:** DeepSeek provides technical follow-up.
- **Model 3:** Mistral simplifies the answer for a 10-year-old.

💡 This is helpful in research, validation, and generating multiple perspectives.

Error Handling

- Invalid or expired API keys are caught and displayed.
 - The app will **skip to the next available key** for OpenRouter if rate-limited.
 - Helpful messages like:
 OpenRouter Error 401: Unauthorized
-

Deployment Options

You can host this app using:

-  **Localhost:** `streamlit run app.py`
 -  **Render.com:** Add `render.yaml`
 -  **Streamlit Community Cloud**
 -  **Vercel** (if migrating to FastAPI backend later)
-

Future Enhancements

- Save chat history to file (PDF/Markdown)
- Token cost estimation panel
- Image input for vision models

- Model latency and usage graph
 - Export conversations
-

Best Practices

- Keep `.env` secure.
 - Check quota and limits of each LLM provider.
 - Use chaining only when necessary (can be costly in API tokens).
 - Log API responses if needed for debugging (disabled by default).
-

License

MIT License © 2025 NemHem AI

Contact & Support

-  Email: `your.email@example.com`
 -  GitHub: github.com/yourusername
 -  Project Page: *coming soon*
-

Let me know if you'd like this exported as a **PDF** or **Google Docs link-ready version**, and I'll help format it or share a downloadable file.